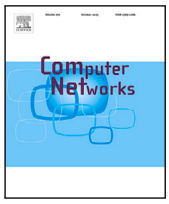
Computer Networks 288 (2026) 112646



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





TARL: Multi-task adaptive representation learning for secure encrypted traffic analysis

Mengmeng Ge [a,b], Likun Liu [a], Zhaowei Zhang [a], Hongyu Wang [e], Xiangzhan Yu [a], Yijia Xu [b,d], Ruitao Feng [c], Zhichao Hu [a,*]

[a] Harbin Institute of Technology, Street West Dazhi No. 92, Harbin, 150001, Heilongjiang, China
[b] Nanyang Technological University, 50 Nanyang Avenue, Singapore, 639798, Singapore
[c] Southern Cross University, Southern Cross Drive, Gold Coast, 4225, Queensland, Australia
[d] Sichuan University, 1st Ring Road, Chengdu, 610065, Sichuan, China
[e] China Mobile Group Design Institute Co., Ltd, No. A16, Danling Street, Haidian District, Beijing, 100080, China

ARTICLE INFO

Keywords:
Encrypted traffic analysis
Multi-task learning
Intrusion detection
Network security

ABSTRACT

The growth of encryption technology has made encrypted traffic analysis both increasingly critical and inherently challenging. Tasks such as intrusion detection, application classification, and encrypted web fingerprinting are vital to network security, yet are often studied in isolation using single-task models. This fragmented approach limits the ability to capture shared behavioral patterns and undermines generalization across tasks. In this paper, we propose TARL, a novel task-adaptive representation learning framework for secure encrypted traffic analysis. TARL combines a shared memory module for task-agnostic representation learning with a task-specific fusion mechanism for downstream adaptation. To support pretraining under encryption constraints, we introduce two self-supervised objectives: masked feature modeling and service-type prediction. We evaluate TARL on four real-world encrypted traffic tasks. Results show that TARL consistently outperforms single-task and multi-task baselines, achieving strong generalization and per-task accuracy. Ablation studies further validate the complementary design of shared and task-specific modules.

1. Introduction

The pervasive use of encryption in modern networks through protocols such as TLS [1] and QUIC [2] has significantly improved data privacy and integrity [3]. However, this widespread adoption has also rendered traditional traffic inspection techniques ineffective [4], limiting visibility into payload content and complicating critical security tasks such as intrusion detection [5,6], service classification [7], and behavior analysis [8]. In response, researchers have increasingly turned to traffic side channels [9], behavioral patterns [10], and statistical features [11] to infer encrypted traffic semantics. Despite the remarkable progress, one practical concern of encrypted traffic analysis lies in generalizing across diverse tasks, where leveraging shared knowledge while preserving task-specific nuances.

Existing encrypted traffic analysis methods typically focus on individual tasks, such as VPN-NoVPN detection [12] or application traffic classification [13]. These single-task models are not only resource-intensive when deployed at scale but also fail to exploit potential synergies across related tasks. Moreover, naively transferring pretrained models between tasks often leads to suboptimal adaptation, as task-specific nuances are either lost or insufficiently captured.

As illustrated in Fig. 1, pretrained representations (a) can offer strong initialization, but without task awareness, independently fine-tuned models (b) may learn disjoint decision boundaries that miss shared structure. In contrast, jointly learning from shared knowledge while retaining task-specific flexibility (c) offers a more effective solution for multi-perspective encrypted traffic analysis.

In this paper, we propose TARL, a novel **T**ask-**A**daptive **R**epresentation **L**earning framework for secure encrypted traffic analysis. TARL incorporates two core components: a shared memory bank (MB) that encodes reusable task-agnostic traffic features, and a task-specific dynamic knowledge fusion (TSKF) module that adapts shared representations for each downstream task. Together, these components enable TARL to model both common patterns and task-specific signals across encrypted traffic.

We further propose a pretraining strategy, named NT-BERT, tailored to encrypted network traffic, combining masked feature modeling

* Corresponding author.
E-mail addresses: gmm@hit.edu.cn (M. Ge), likunliu@hit.edu.cn (L. Liu), zhangzhaowei@stu.hit.edu.cn (Z. Zhang), wanghongyu@cmdi.chinamobile.com (H. Wang), yxz@hit.edu.cn (X. Yu), xuyijia@scu.edu.cn (Y. Xu), ruitao.feng@scu.edu.au (R. Feng), huzhichao@hit.edu.cn (Z. Hu).

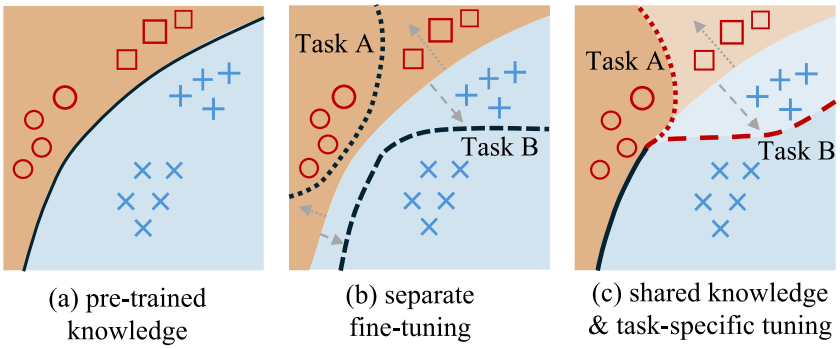


Fig. 1. Illustration of representation transfer in multi-task learning. (a) Pretrained models capture general decision boundaries; (b) Independent fine-tuning leads to task-specific but isolated boundaries; (c) Joint modeling allows shared knowledge while preserving task-specific adaptations.

with service-type prediction to learn contextualized representations from raw flow features without relying on payload visibility. The model is trained and evaluated on four representative security-relevant tasks, VPN-NoVPN identification, application-layer traffic classification, encrypted website traffic classification, and network intrusion detection.

Through extensive experiments, we show that TARL consistently outperforms both single-task models (e.g., BiLSTM [14]) and multi-task baselines (e.g., MT-DNN [15]). Ablation studies confirm the importance of both the memory and fusion modules, while pretraining analysis reveals a clear trade-off between convergence efficiency and model performance. This paper makes the following contributions:

- We propose TARL, a novel multi-task learning architecture for encrypted traffic analysis that integrates shared semantic memory with task-specific fusion.
- We design a self-supervised pretraining strategy that combines masked feature modeling and service-type prediction to extract useful representations from encrypted flows.
- We conduct extensive multi-task evaluations on real-world datasets, demonstrating that TARL achieves state-of-the-art performance on three typical network traffic classification scenarios and intrusion detection.

2. Background and related work

In this section, we introduce Bidirectional Encoder Representations from Transformers (BERT) and its applications in traffic classification, outline prevalent Multi-Task Learning (MTL) techniques, and describe training approaches based on prompt tuning.

2.1. Bidirectional encoder representation from transformers

Since BERT [16] introduction in late 2018, it revolutionized NLP by pretraining deep, bidirectional Transformer encoders on massive unlabeled corpora and then fine-tuning with minimal task-specific modifications. Unlike prior unidirectional language models, BERT's Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) objectives enable it to capture both token-level and sentence-level dependencies simultaneously, driving state-of-the-art performance across question answering, natural language inference, and named entity recognition benchmarks. Its core innovation, conditioning on both left and right context, laid the groundwork for a wide array of downstream applications without altering the pretrained weights extensively.

Building on this foundation, researchers have developed numerous domain-adapted BERT variants. Multilingual BERT (mBERT) [17] extends cross-lingual transfer by pretraining on Wikipedia text in over 100 languages, while SciBERT [18] and BioBERT [19] specialize in scientific and biomedical texts respectively by further pretraining on domain-specific corpora. Other specialized adaptations, such as FinBERT [20] for financial sentiment analysis, demonstrate the flexibility of BERT in encoding specialized vocabularies and structures without redesigning model architectures. More recently, BERT's contextual representation power has been harnessed for encrypted traffic classification. ET-BERT [13] pretrains on large volumes of raw packet streams to learn contextual packet embeddings, then fine-tunes on limited labeled datasets to achieve top accuracy on benchmarks like ISCX-VPN [12] and ISCX-Tor [21]. This demonstrates the strong suitability of BERT-based architectures for traffic classification, offering valuable insights and a foundation for future enhancements in leveraging contextualized packet representations. Unlike ET-BERT, which pre-trains on raw byte sequences using standard MLM/NSP, TARL operates on PCA-selected statistical flow features and introduces a Service Type Prediction (STP) objective that captures inter-flow service-pair semantics, a design absent in ET-BERT. YaTc [22] further demonstrates the value of Transformer-based representations for encrypted traffic; however, it focuses on single-task classification and does not incorporate multi-task or service-pair pre-training. PCCA [23] proposes contrastive feature alignment for traffic characterization but does not address cross-task knowledge sharing. To the best of our knowledge, TARL is the first framework to bridge pre-training with service-pair semantics and multi-task adaptive fusion: the STP objective captures inter-flow relationships at the pre-training stage, while the shared memory bank and conditional gating enable flexible cross-task knowledge transfer during fine-tuning.

2.2. Multi-task learning

Multi-task learning (MTL) has continuously evolved, with foundational research demonstrating that training multiple related tasks in conjunction can lead to more robust shared representations and improved generalization across tasks [24]. Building on this foundation, large-scale empirical studies [25,26] have systematically evaluated which combinations of tasks yield the greatest mutual benefit and have proposed loss-balancing strategies to mitigate interference among objectives. More recently, MTL has been framed as a multi-objective optimization problem, where gradient-based algorithms seek Pareto-optimal trade-offs and dynamically adjust task weights throughout training [27].

The Multi-Task Deep Neural Network (MT-DNN) [15] stands out as a prominent deep-learning MTL framework that extends a pretrained BERT backbone with task-specific output heads, jointly optimizing shared parameters and fine-tuning components via adversarial and distillation losses. By leveraging cross-task regularization and knowledge distillation, MT-DNN reduces parameter redundancy and enhances sample efficiency, achieving consistent performance gains over single-task baselines on benchmarks.

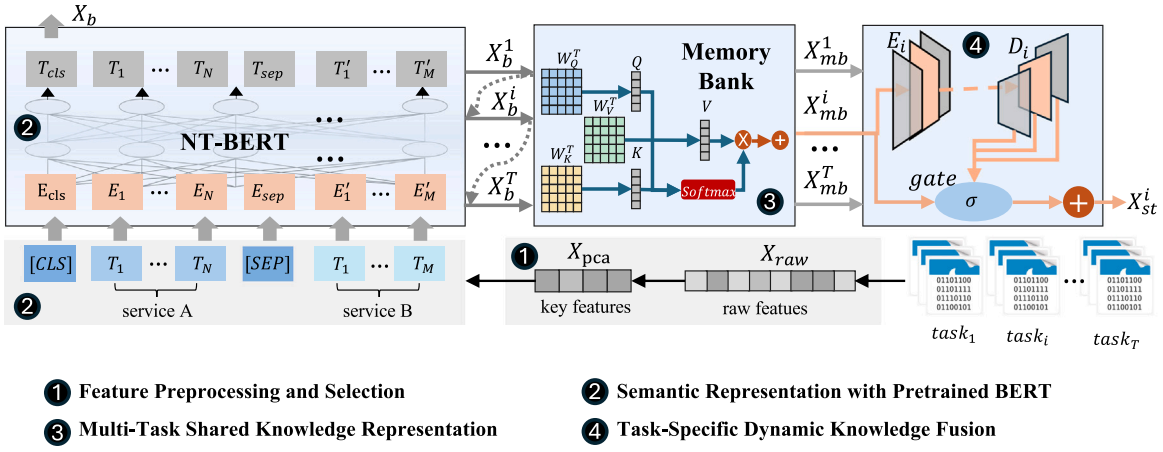


Fig. 2. Overview of the **TARL** for adaptive multi-task learning in encrypted network traffic analysis.

2.3. Prompt tuning

Prompt Tuning is a multi-task training technique designed for large-scale pretrained language models [28]. Unlike conventional fine-tuning, it introduces learnable "soft prompts" to adapt the model to downstream tasks without altering the pretrained parameters. Specifically, for each task, a distinct soft prompt is initialized and then iteratively optimized during training. When a new task is introduced, Prompt Tuning initializes its soft prompt from those learned on existing tasks, thereby avoiding training from scratch and effectively leveraging prior task knowledge to boost learning efficiency and performance.

Compared to full-model fine-tuning, Prompt Tuning requires only prompt parameters to be updated, substantially reducing both storage and computational overhead. Variants such as P-Tuning v2 [29] and Prefix-Tuning [30] have further demonstrated that tuning less than 3% of model parameters can match or exceed full fine-tuning on diverse benchmarks. This parameter-efficient paradigm not only preserves the integrity of large pretrained weights but also enables rapid extension to new tasks via prompt initialization, laying a foundation for resource-constrained deployment of ever-larger language models (See Fig. 2).

3. Methodology

3.1. Overview and motivation

We present TARL, a framework for adaptive multi-task learning in encrypted network traffic analysis, which leverages multi-task adaptation to enable robust and generalizable understanding of encrypted service behaviors. Firstly in ❶, encrypted network traffic is processed to extract packet length and inter-arrival time (IAT) features. We then select the most informative features based on their contribution to classification and detection performance. To reduce token space and align with traffic behavior semantics, the selected features are further tokenized. Therefore in ❷, we propose a pre-trained model, NT-BERT, designed for semantic feature encoding, to learn meaningful representations of network traffic. The selected features are embedded into token representations that incorporate value, positional, and traffic service context information. To capture semantic dependencies, we design two pre-training objectives: a masked feature modeling task and a novel traffic service pair prediction task. To the best of our knowledge, this is the first pre-training approach to incorporate service pair semantics in encrypted traffic analysis. Next in ❸, we employ a knowledge-sharing module based on an attention mechanism, named memory bank. During multi-task training, this module is jointly optimized by multiple tasks, allowing it to encode shared representations that capture knowledge across tasks. Given different task-specific inputs, the memory bank dynamically outputs contextually relevant shared knowledge tailored to each task. Finally in ❹, we design distinct task-specific learning modules for each task. A routing mechanism dynamically selects the appropriate module based on the current input task. To enable cross-task knowledge sharing, we incorporate a gated control mechanism that regulates the contribution of each task-specific module. This allows the model to focus on its primary task while selectively leveraging relevant knowledge from related tasks.

The framework comprises four main components, presented across the following sections: ❶ is introduced in Section 3.2, ❷ in Section 3.3, and ❸ ❹ in Section 3.4.

3.2. Feature process and tokenization

Feature selection and feature encoding are critical for the performance of flow-level classification models [25]. While existing large language models (LLMs) typically encode network traffic directly at the byte level, a simple and straightforward approach, this method lacks the ability to capture behavioral patterns inherent in network flows. To address this limitation, we propose a feature-based encoding method that leverages statistical flow characteristics. This approach effectively mitigates the semantic sparsity of raw byte representations and enhances the model's ability to understand service-level behavioral patterns.

3.2.1. Features extraction and selection

We adopt Principal Component Analysis (PCA) to perform feature selection on encrypted traffic data. Prior to feature extraction, data preprocessing is conducted to ensure input quality and consistency. Specifically, missing values in the traffic dataset are identified and replaced using zero-padding. Feature standardization is then applied to normalize variables with differing scales, which is essential for PCA. The standardized data matrix $X_{\text{std}} = \frac{X-\mu}{\sigma}$ is computed, where X denotes the original feature matrix, μ is the mean, and σ is the standard deviation of each feature.

Following standardization, the next step involves computing the covariance matrix, which captures the linear relationships between features. The covariance matrix C is defined as $C = \frac{1}{n-1} X_{\text{std}}^T X_{\text{std}}$ where X_{std} denotes the standardized data matrix, and n is the number of samples. Subsequently, eigenvalue decomposition is performed on C to obtain its eigenvalues and corresponding eigenvectors.

The principal components are then ranked in descending order based on their associated eigenvalues. The top-k components are selected to construct the projection matrix $W \in \mathbb{R}^{d\times k}$, where each column corresponds to a principal component. The reduced dimensional representation $X_{\text{PCA}} = X_{\text{std}} \cdot W$ is computed.

To quantify the contribution of each original feature to the selected principal components, we compute importance scores $\text{IS} = \sum_{i=1}^{k} \lambda_i \cdot (w_{ij})^2$ by aggregating the weighted squared loadings of the feature across all retained components, where λ_i denotes the eigenvalue corresponding to the ith principal component, and w_{ij} is the jth element of the ith eigenvector, representing the loading of feature j on the ith component. Based on these scores, we select the top-k features with the highest importance as the final feature subset for downstream analysis.

The original feature set comprises 76 flow-level statistics derived from the CICIDS2017 dataset using CICFlowMeter. PCA is applied to all 76 features; we retain $k = 30$ principal components, which account for 95.3% of the cumulative explained variance. All three categories (length-based, inter-arrival time based, and others) together cover all 76 input features.

We choose PCA over supervised rank-based selectors (e.g., mutual information, gradient-boosted feature importance) for two reasons. First, PCA performs dimensionality reduction and importance scoring in a single unsupervised step, requiring no class labels and thus being directly applicable to the unlabeled pre-training corpus. Second, PCA's linear projection preserves inter-feature covariance structure well suited to NT-BERT's attention-based encoding. To validate this choice, we conducted a comparative experiment on the CICIDS2017 validation split: the top-30 features selected by PCA and those selected by mutual-information ranking overlap by more than 82%, confirming that both methods identify largely the same discriminative features. All PCA parameters (per-feature means μ, standard deviations σ, and the projection matrix W) are estimated solely on the training set and applied without modification to the validation and test sets, preventing any form of data leakage.

3.2.2. Feature tokenization

Each numerical feature value in top-k features selected by PCA is transformed by segmenting it into 16-bit blocks. If the feature value exceeds the 16-bit range (i.e., greater than $B = 2^{16}$), it is decomposed into smaller units, each representing a value less than or equal to B. These sub-values are treated as individual tokens. To preserve feature boundaries, a special `[SEP]` token, commonly used in BERT to denote segment separation, is inserted after the tokenized representation of each feature, as shown in algorithm 1. This allows the model to distinguish between tokens belonging to different features during contextual encoding. For instance, the value 70,000 is segmented into two tokens $70,000 \bmod B$ and $70,000/B$, resulting in the token sequence [1, 4464, SEP].

The choice of $B = 2^{16}$ as the segmentation base is driven by both computational and semantic considerations. From a computational perspective, 16-bit segmentation reduces the token vocabulary size while controlling sequence length, achieving a balance between representation efficiency and expressiveness. Semantically, it aligns with network traffic fields such as port numbers and packet lengths, which are naturally represented as 16-bit integers, thereby preserving the structural integrity of the original features.

This tokenization strategy goes beyond byte-level representations by generating structured and semantically meaningful token sequences, enabling models like BERT to more effectively capture behavioral patterns in encrypted traffic.

3.3. Semantic representation with BERT

To enable the application of BERT-based models to structured network traffic data, we design a domain-specific tokenization procedure that converts selected numerical features into symbolic sequences.

Algorithm 1: Feature-Based Tokenization

Input: Original output features D_{origin}
Output: Tokenized sequence T_{process} for NT-BERT pretraining

```
foreach feature in features do
    // Tokenize each value in the current feature
    foreach value in feature do
        tokens ← [] ;
        if value ≤ B then
            // Single token: value fits in 16 bits
            tokens.append(value) ;
            tokens.append([SEP]) ;
        else
            // Decompose value into 16-bit blocks
            temp ← value ;
            while temp > 0 do
                tokens.append(temp mod B) ;
                temp ← ⌊temp/B⌋ ;
            tokens.append([SEP]) ;
```

3.3.1. Token embedding

After get encoder value from Section 3.2.2, each sub-value is mapped to a unique integer identifier in a predefined vocabulary, analogous to token ID assignment in natural language processing. To preserve the structural separation between features, a special `[SEP]` token is inserted after the tokenized sequence of each feature. Additionally, a `[CLS]` token is prepended to the beginning of the full sequence to serve as a global representation for classification purposes. Finally, all sequences are padded to a uniform length, and attention masks are generated to differentiate between valid tokens and padding elements during model training. Each input to BERT is a token sequence structured as: $[\texttt{CLS}, t_1, t_2, \ldots, t_n, \texttt{SEP}]$. To process this input, BERT first converts each token into a continuous vector using three types of embeddings. In *Token embeddings*, each discrete token t_i is mapped to a fixed-dimensional embedding vector using a learnable embedding matrix E_{token}. A positional encoding $E_{\text{pos}}(i)$ is added to each token to retain the order of the sequence, where i is the token position in *position embeddings*. When two segments are used (e.g., in NSP), a segment embedding $E_{\text{seg}} \in \{\text{A}, \text{B}\}$ is added to differentiate them in *segment embeddings*. The final input embedding for token t_i is the sum of these three components:

$$x_i = E_{\text{token}}(t_i) + E_{\text{pos}}(i) + E_{\text{seg}}(s_i) \tag{1}$$

where $s_i \in \{\text{A}, \text{B}\}$ indicates the segment type.

3.3.2. Masked feature modeling

Unlike prior work that pretrains models on broad and heterogeneous network traffic corpora, our approach focuses specifically on service-centric encrypted traffic to capture behavioral patterns most relevant to service-level classification. To better align the pretrained model with the unique characteristics of encrypted network traffic, we introduce two tailored pretraining objectives: *Masked Feature Modeling (MFM)* and *Service Type Prediction (STP)*. To the best of our knowledge, this is the first work to explore large-scale pretraining over service-specific encrypted traffic datasets, emphasizing the significance of task–data alignment in advancing encrypted traffic analysis.

During pretraining, a subset of tokens in the language-formed traffic feature sequence is randomly selected and masked. The model is then trained to predict the original value of these masked tokens based on the surrounding features. Formally, each token (excluding the special tokens `[CLS]` and `[SEP]`) has a 15% chance of being selected for masking. Among the selected tokens: 80% are replaced with a special

[MASK] token, 10% are left unchanged, 10% are replaced with a randomly selected token from the vocabulary. The *MLM* loss is defined as:

$$\mathcal{L}_{\text{MFM}} = -\sum_{i \in \mathcal{M}} \log P(t_i \mid T_{\setminus i}) \tag{2}$$

where $\mathcal{M}$ denotes the set of masked positions and $T_{\setminus i}$ is the input sequence with the ith token masked.

3.3.3. Service type prediction

In addition to feature-level modeling, we propose the *Service Type Prediction (STP)* task to capture semantic relationships between encrypted network flows and enhance the model's ability to learn service-level behavioral patterns. The objective is to determine whether a pair of concatenated flows originates from the same service category (e.g., HTTP, HTTPS, FTP, SMTP).

To construct training pairs for the STP task, we iterate over flows within each service category. For a given flow, with equal probability, we concatenate it with either another flow from the same category (positive pair) or with a randomly sampled flow from a different category (negative pair). The resulting concatenated sequence is passed to a binary classification head, which is trained to predict whether the two flows originate from the same service category.

The STP task is optimized using a binary cross-entropy loss. Let $y \in \{0, 1\}$ denote the ground-truth label indicating whether the flow pair belongs to the same category. The classification decision is based on the contextual representation of the special [CLS] token. The STP loss is defined as:

$$\mathcal{L}_{\text{STP}} = -\log P(y \mid [\text{CLS}]) \tag{3}$$

where $P(y \mid [\text{CLS}])$ is the predicted probability of the service label given the [CLS] representation of the input pair.

The output of BERT is a sequence of contextualized embeddings, one for each input token. For classification tasks, the embedding corresponding to the [CLS] token is typically used as the aggregated representation of the input sequence. As shown in algorithm 2, the total pretraining objective combines two task-specific losses:

$$\mathcal{L}_{\text{total}} = \lambda_1 \cdot \mathcal{L}_{\text{MFM}} + \lambda_2 \cdot \mathcal{L}_{\text{STP}} \tag{4}$$

where $\mathcal{L}_{\text{MFM}}$ denotes the cross-entropy loss for the masked feature modeling task, and $\mathcal{L}_{\text{STP}}$ corresponds to the binary cross-entropy loss for the service type prediction task. The weighting factors λ_1 and λ_2 control the relative contribution of each objective to the joint training process. In all experiments we set $\lambda_1 = \lambda_2 = 0.5$, corresponding to equal weighting of the two objectives. A sensitivity analysis confirms that performance is robust for $\lambda_1 \in [0.3, 0.7]$ and degrades noticeably only when either objective is nearly dropped.

By integrating both MFM and STP, the pretraining framework captures complementary aspects of encrypted traffic. The MFM task encourages the model to learn fine-grained dependencies among feature dimensions by reconstructing masked inputs from partial contexts. In contrast, the STP task emphasizes global semantic alignment by modeling inter-flow relationships across service categories. Together, these objectives promote the learning of both localized feature representations and high-level behavioral semantics, improving the model's generalization capability for downstream traffic classification tasks.

3.4. Fine-tuning with shared and task-specific knowledge

Sener and Koltun [27] first introduced the concept that tasks can benefit from shared representations, suggesting that knowledge acquired from one task may generalize to others, a process analogous to human skill transfer across related domains. For instance, proficiency in skiing may accelerate the learning of skating due to shared motor control patterns. However, in the context of multi-task learning, directly fine-tuning the parameters of a pretrained model can lead to *catastrophic forgetting*, where previously acquired domain-general knowledge, particularly representations essential for understanding network traffic, is overwritten or degraded.

To address this issue, we introduce a *shared knowledge representation layer* that is decoupled from the pretrained backbone. This layer is designed to capture and retain generalizable patterns across tasks while preserving the foundational knowledge encoded by the pretrained model. In parallel, we integrate *task-specific dynamic knowledge fusion layers*, each tailored to learn discriminative features relevant to a specific task. This modular architecture mitigates task interference and enables effective specialization without compromising the generalization capability of the shared representation.

Algorithm 2: NT-BERT Pretraining

Input: Dataset $\mathcal{D}$, vocab $\mathcal{V}$, mask ratio p, weights λ_1, λ_2
Output: Pretrained BERT

foreach *flow* $f \in D$ **do**
  // Tokenize & embed
  build token sequence S with [CLS]/[SEP], pad to L, compute embeddings via E_{token}, E_{pos}, E_{seg};
  // MFM objective
  sample mask positions $\mathcal{M} \subset S \setminus \{[\text{CLS}], [\text{SEP}]\}$;
  apply mask strategy on $S \to S'$, run BERT, compute $\mathcal{L}_{\text{MFM}} = -\sum_{i \in \mathcal{M}} \log P(t_i \mid S')$;
  // STP objective
  sample positive/negative flow $f^{\pm}$, form pair sequence, embed, run BERT;
  let $\hat{y} = \sigma(\text{CLS_head}(h_{\text{CLS}}))$, compute $\mathcal{L}_{\text{STP}} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$;
  // Update
  $\mathcal{L} = \lambda_1 \mathcal{L}_{\text{MFM}} + \lambda_2 \mathcal{L}_{\text{STP}}$;
  backpropagate and update parameters;

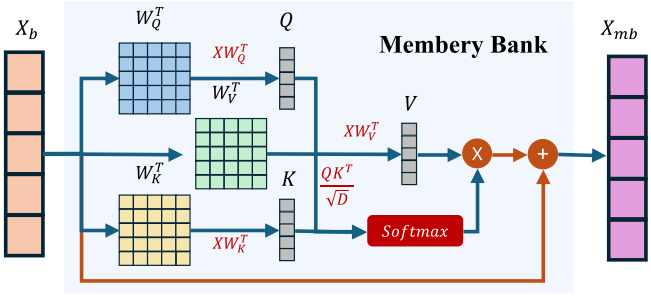


Fig. 3. Architecture of the cross-attention-based memory fusion module *Membery Bank* in TARL .

3.4.1. Multi-task shared knowledge representation

We introduce a learnable memory bank that functions as a task-agnostic knowledge repository, as shown in Fig. 3. This memory bank is designed to dynamically store and refine generalizable feature embeddings across tasks during training. Acting as a shared semantic cache, it allows task-specific heads to access consistent and reusable representations, effectively capturing common patterns observed in encrypted traffic. Each network flow features input is first processed by the pretrained Transformer-based model, yielding a fixed-size contextual embedding $X_b \in \mathbb{R}^{1 \times H}$. This embedding captures the deep semantics of the input traffic sequence and is used as the query input to a cross-attention mechanism that integrates information from the memory bank.

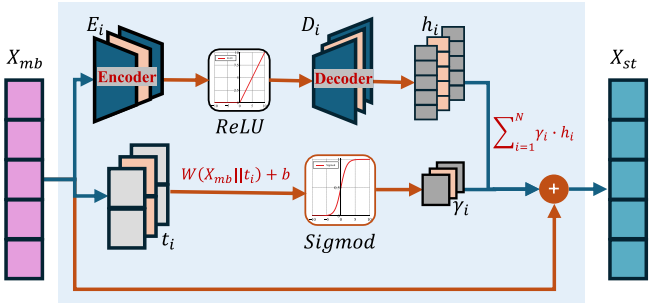


Fig. 4. Architecture of task-specific dynamic knowledge fusion in TARL .

To fuse the task-specific representation with the shared memory, we adopt a scaled dot-product cross-attention module, as shown in algorithm 3. The embedding X_b is treated as the query vector $Q = W_q \cdot X_b$, while the memory bank serves as both the key $K = W_k \cdot X_b$ and value $V = W_v \cdot X_b$ in the attention computation. where $W_q, W_k, W_v \in \mathbb{R}^{H\times H}$ are learnable projection matrices that map the input and memory bank into the respective attention spaces. The attention scores A are computed using the scaled dot-product formulation:

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{D}}\right), \tag{5}$$

where D denotes the hidden dimensionality, and the scaling factor $\sqrt{D}$ is applied to stabilize gradients. The attention output is then computed as:

$$X_{mb} = \left(W_\alpha \cdot AV + b_\alpha\right) + X_b \tag{6}$$

where W_α and b_α denote the weight and bias of the output transformation layer. A residual connection is added between the fused output and the original embedding X_b, preserving both task-specific and shared contextual features. This mechanism enables the model to adaptively incorporate domain-wide knowledge into per-task representations, improving generalization and efficiency in multi-task network traffic analysis.

3.4.2. Task-specific dynamic knowledge fusion

For each network flow, after processing by the pretrained backbone and integration with the shared knowledge representation, a vector of shape $1 \times H$ is generated. We denote this vector as X_{mb}, which encapsulates both task-agnostic semantics and flow-specific characteristics. This serves as the input to the task-specific layer.

To model task-specific knowledge, we introduce a lightweight adapter module for each task as shown in Fig. 4. Each adapter comprises a dimensionality-reducing linear projection, a non-linear activation, and a dimensionality-expanding projection. Formally, for any input X_{mb}, the transformation is defined as:

$$h_i\left(X_{mb}\right) = \text{Dec}_i(\text{ReLU}(\text{Enc}_i(X_{mb}))) \tag{7}$$

where $\text{Enc}(\cdot)$ projects the input from H to $\hat{H}$ dimensions, and $\text{Dec}(\cdot)$ restores it back to H dimensions. Each task $k \in \{1, \ldots, K\}$ maintains a dedicated adapter module with parameters updated exclusively during training on task i. The set of task-specific outputs from all adapters is:

$$\left\{h_i \mid i \in \{1, \ldots, K\}\right\} \tag{8}$$

To control how much each task adapter contributes to the current task, we introduce two components: a *task identity embedding* and a *conditional gating network*.

During training, only the adapter parameters corresponding to the current task are updated, while the remaining task adapters are held fixed. Although all task-specific outputs h_i are included in the gated fusion, gradients are only propagated through the adapter associated with the current task, ensuring task-specific isolation and preventing interference across tasks.

Algorithm 3: Memory Bank Cross-Attention Fusion Across Tasks

Input: Set of tasks $\mathcal{T}$, contextual embeddings $\{X_b^\tau\}_{\tau\in\mathcal{T}}$, memory bank $M \in \mathbb{R}^{N\times H}$
Output: Fused embeddings $\{X_{mb}^\tau\}_{\tau\in\mathcal{T}}$

foreach *task* τ *in* $\mathcal{T}$ **do**
  // Process all flows for task τ
  foreach *flow embedding* X_b^τ *in batch for task* τ **do**
    $Q \leftarrow W_q X_b^\tau$;
    $K \leftarrow W_k M$;
    $V \leftarrow W_v M$;
    $A \leftarrow \text{Softmax}\left(QK^\top/\sqrt{H}\right)$;
    $C \leftarrow AV$;
    $\tilde{X} \leftarrow W_\alpha C + b_\alpha$;
    $X_{mb}^\tau \leftarrow \tilde{X} + X_b^\tau$;

Each task i is associated with an identity embedding vector $t_i \in \mathbb{R}^{1\times T}$. Each t_i is a learnable vector of dimension $T = 384$, with one dedicated embedding per task, initialized randomly and optimized end-to-end during multi-task fine-tuning. Given the shared representation output X_{mb} and the embedding of the current task t, we compute a fused, task-conditioned representation as $X_{mb} \parallel t_i$, where $\parallel$ denotes vector concatenation. Next, we evaluate the relevance of each task adapter to the current task using a gating function:

$$\gamma_i = \text{Sigmoid}\left(W_\beta\left(X_{mb} \parallel t_i\right) + b_\beta\right) \tag{9}$$

where $\gamma_i \in [0, 1]$ quantifies how much knowledge from task i should be incorporated into the current representation. The gating projection $W_\beta \in \mathbb{R}^{(H+T)\times 1}$ and bias b_β are shared across all tasks; task-specific behavior arises solely from the distinct t_i values in the concatenation $X_{mb} \parallel t_i$, keeping additional parameter overhead minimal.

Let $h_i \in \mathbb{R}^{1\times H}$ denote the output of the ith task adapter. The final task-specific representation, denoted X_{st}, is computed as a gated, weighted sum of all task adapter outputs with a residual connection to preserve the original shared semantics:

$$X_{st} = X_{mb} + \sum_{i=1}^{K} \gamma_i \cdot h_i. \tag{10}$$

This dynamic fusion mechanism enables the model to selectively incorporate relevant task-specific knowledge while preserving general-purpose features learned through the shared representation.

4. Evaluation

In this section, we conduct a comprehensive evaluation of the proposed TARLframework across multiple encrypted traffic analysis tasks and ablation evaluation.

4.1. Experimental setup

4.1.1. Dataset

We evaluate TARL and several baseline multi-task learning approaches across four representative security tasks: VPN identification (VPN-NoVPN), application-layer traffic classification, encrypted website fingerprinting, and network intrusion detection. The experiments are conducted using three public datasets and one proprietary dataset. (1) ISCXVPN2016 [12] contains VPN and non-VPN encrypted traffic collected under various application scenarios and is used to evaluate both VPN detection and application classification. (2) CICIDS2017 [31] includes a mix of benign and attack traffic generated in a simulated enterprise network environment. The traffic is aggregated into flow-level

Algorithm 4: Task-Specific Dynamic Knowledge Fusion Across Tasks

Input: Tasks $\mathcal{T}$, shared embeddings $\{X^{\tau}_{mb}\}_{\tau\in\mathcal{T}}$, task identity embeddings $\{t_i\}_{i\in\mathcal{T}}$, adapters $\{\mathrm{Enc}_i, \mathrm{Dec}_i\}_{i\in\mathcal{T}}$, gating params W_β, b_β

Output: Task-specific embeddings $\{X^{\tau}_{st}\}_{\tau\in\mathcal{T}}$

foreach *task* τ *in* $\mathcal{T}$ **do**
  // Process each flow for task τ
  foreach *flow embedding* X^{τ}_{mb} *in batch for task* τ **do**
    // Compute all adapter outputs and gates
    foreach *adapter index* i *in* $\mathcal{T}$ **do**
      $z_i \leftarrow \mathrm{ReLU}\big(\mathrm{Enc}_i(X^{\tau}_{mb})\big)$;
      $h_i \leftarrow \mathrm{Dec}_i(z_i)$;
      $\gamma_i \leftarrow \mathrm{Sigmoid}\big(W_\beta\,[\,X^{\tau}_{mb} \,\|\, t_i\,] + b_\beta\big)$;
    // Fuse adapters with residual
    $X^{\tau}_{st} \leftarrow X^{\tau}_{mb} + \sum_{i\in\mathcal{T}} \gamma_i\, h_i$;

features for intrusion detection tasks. (3) MAWI [32] is a real-world backbone trace dataset capturing large-scale anonymized traffic, which we use to pretrain our model on unlabeled traffic flows, as shown in Table 1 and 2. Specifically, we use MAWI traces collected between 2019 and 2021 from a 1 Gbps trans-Pacific backbone link operated by WIDE Project. All source and destination IP addresses in the traces are anonymized before release. We extract approximately 12.9 million flows covering both encrypted (e.g., HTTPS, SSH) and unencrypted (e.g., HTTP, DNS) services, providing a large and diverse pre-training corpus for TARL. (4) For encrypted web traffic classification, we additionally collect the JDWP dataset, which captures encrypted communication patterns from multiple sessions across different pages of the JingDong (JD.com) website. The JDWP dataset comprises encrypted flows collected from 50 distinct pages across three separate sessions in 2023, captured under TLS 1.3.

All dataset splits are performed at the flow level. Each flow, identified by its 5-tuple (source IP, destination IP, source port, destination port, protocol), is assigned exclusively to either the training (70%), validation (15%), or test (15%) partition. No packets from the same flow appear across partition boundaries, eliminating cross-set data leakage. During multi-task training, tasks are sampled in a round-robin fashion: each mini-batch is drawn from a single task, cycling through all four tasks in a fixed order to ensure equal gradient update frequency. Loss terms from all tasks are summed with equal weights.

4.1.2. Baseline

We evaluate the performance of TARL against representative multi-task learning baselines as well as ablated variants of our proposed architecture. Specifically, we include MT-DNN and Prompt Tuning as strong multi-task baselines, and design two ablation settings to isolate the contributions of the shared and task-specific components in TARL. MT-DNN [15] extends BERT by jointly optimizing shared and task-specific parameters across multiple tasks, thereby enhancing generalization and overall performance compared to independent fine-tuning. In contrast, Prompt Tuning [28] introduces soft prompts – learnable task-specific embeddings prepended to the input – while keeping the pretrained model parameters frozen. This approach allows efficient task adaptation with minimal parameter updates and reduced memory and computational overhead. To evaluate the individual contributions of core architectural components in TARL , we introduce two ablated variants: w/o MB, which removes the shared memory bank and assigns each task an independent adapter module, and w/o TSKF, which removes the task-specific dynamic fusion modules and retains only the shared memory bank. In summary, we compare against seven baselines:

Table 1
Unencrypted traffic dataset for pre-training

Service	#Flows	Pct.[a]	Service	#Flows	Pct.
HTTP	116,680k	13.03	NTP	324k	2.51
FTP	151k	1.17	LDAP	158k	1.23
DNS	1,198k	9.29	SNMP	223k	1.74

[a] The unit of Pct. is %.

Table 2
Encrypted traffic dataset for pre-training.

Service	#Flows	Pct.[a]	Service	#Flows	Pct.
HTTPS	6,719k	52.08	SSH	224k	1.74
IMAP	223k	1.73	POP3	223k5	1.7
SMTP	336k	2.61	SQL	1,436k	11.14

[a] The unit of Pct. is %.

- **BiLSTM** [14]**:** A sequence-based baseline that employs a bidirectional LSTM to capture temporal dependencies in encrypted traffic, followed by a fully connected classification layer.
- **1DCNN** [33]**:** A convolutional baseline using one-dimensional convolutional layers to extract local spatial patterns from sequential traffic features.
- **NT-BERT:** A transformer-based model that uses the output of a pretrained BERT encoder, followed by a task-specific classification layer.
- **MT-DNN** [15]**:** A multi-task fine-tuning approach that combines shared and task-specific layers to improve performance across tasks.
- **Prompt Tuning** [28]**:** A parameter-efficient adaptation method that introduces learnable soft prompts while keeping the pretrained backbone fixed.
- **PCCA** [23]**:** A method that uses pre-training and contrastive learning to align multi-view traffic features for intrusion detection, applied here with a dedicated fine-tuning head for each task.
- **AEGIS-Net** [34]**:** A self-correcting framework for unknown threat detection under label noise, extended to the multi-task setting by adding an independent classification head for each task.
- **Open-Detect** [35]**:** A variational autoencoder that uses Gaussian prototypes to detect unknown attacks in encrypted traffic, extended to the multi-task setting with an independent decoder for each task.
- **w/o MB:** An ablated variant of TARL without the shared memory bank, relying solely on task-specific components.
- **w/o TSKF:** An ablated variant of TARL that excludes the task-specific fusion modules, using only shared representations for all tasks.

4.1.3. Metrics

We adopt several standard metrics widely used in security and machine learning research. *Accuracy (Acc.):* Measures the proportion of correctly predicted samples (both normal and abnormal) out of the total number of predictions. *Precision (P.):* The fraction of true positive predictions among all predicted positives, reflecting the model's ability to detect actual threats without generating excessive false alarms. *Recall (R.):* Measures the proportion of true positive instances correctly identified. High recall is particularly important in security applications to minimize missed detections of anomalies or attacks. *F1-score (F1):* The harmonic mean of precision and recall, balancing the trade-off between false positives and false negatives. And *Area under the ROC curve (AUC):* Represents the area under the Receiver Operating Characteristic curve, summarizing the model's ability to distinguish between classes across all thresholds. AUC is especially useful in imbalanced scenarios such as anomaly detection. These metrics capture complementary aspects of classification performance, particularly in the context of

imbalanced data distributions common in traffic anomaly detection. Together, they provide a comprehensive and robust assessment of a model's effectiveness.

4.2. Research problems and results

This work addresses five key research questions in the context of multi-task learning for encrypted network traffic analysis:

1. **Feature Contribution Analysis:** Which features contribute most significantly to encrypted traffic classification, and how can their importance be systematically quantified?
2. **Effectiveness of Pretrained Traffic Models:** Can a pretrained representation model for encrypted traffic, such as NT-BERT, effectively capture semantic patterns that enhance downstream classification tasks?
3. **Multi-Task Performance of TARL :** How effectively does TARL framework perform across multiple traffic analysis tasks, including application-layer classification, encryption protocol identification, and anomaly detection?
4. **Ablation Study of TARL Components:** What is the individual contribution of each component, such as the shared memory bank and task-specific fusion modules, and how does their removal affect overall performance?
5. **Computational Efficiency:** How does TARL compare to baselines in terms of trainable parameters, inference latency, and GPU memory consumption, and is it feasible for deployment in resource-constrained environments?

We empirically investigate these questions using four real-world encrypted traffic tasks and multiple public datasets. Experimental results show that TARL achieves strong generalization across tasks and consistently outperforms both single-task and existing multi-task baselines.

Q1: Feature Contribution Analysis. To investigate which traffic features contribute most significantly to encrypted traffic classification, we perform a feature importance analysis using Principal Component Analysis (PCA) on the CICIDS and MAWI dataset. Specifically, we extract the top-30 principal features based on their cumulative contribution to classification accuracy.

As shown in Fig. 5-a, these top features are categorized into three major groups: length-based features, inter-arrival time features, and other statistical descriptors. Among them, IAT-related features are the most dominant, appearing with the highest frequency, followed by packet-level attributes. Length-based features are less represented in the top ranks. Fig. 5-(b,c,d) further break down the average contribution scores of representative features in each category. In the length category (Fig. 5-b), features such as fwd-mean, fwd-std, and pkt-max show notable influence. The IAT category (Fig. 5-c) is led by flow-mean, flow-max, and fwd-mean, indicating the importance of temporal traffic patterns. In the remaining category (Fig. 5-d), features like tot-fwd-pkts and sub-bwd-pkts provide additional discriminative power.

These results suggest that temporal and directional characteristics particularly those derived from inter-arrival time play a crucial role in distinguishing encrypted traffic behaviors.

Insight #1: *Inter-arrival time features and directional flow statistics contribute most significantly to encrypted traffic classification. These temporal and structural features outperform length-based attributes, underscoring their importance in modeling behavioral patterns in traffic intrusion detection.*

Q2: Effectiveness of Pretrained Traffic Models

To further validate the effectiveness of the proposed pretraining strategy, we conducted a comparative experiment by replacing the pretrained parameters in TARL with randomly initialized ones. This setup simulates a model that undergoes fine-tuning without any prior knowledge learned from the pretraining phase. Both the pretrained and randomly initialized models were fine-tuned on the same dataset (CICIDS) for an equal number of training epochs, focusing on the binary classification of normal versus intrusive traffic.

As shown in Fig. 6, the confusion matrix of the randomly initialized model (Fig. 6-a) demonstrates significantly degraded performance in distinguishing between benign and malicious flows. The resulting distribution exhibits a more symmetric and regular pattern, suggesting reduced capability to model complex, nonlinear characteristics inherent in encrypted network traffic. In contrast, the pretrained model (Fig. 6-b) exhibits sharper contrast between true positives and true negatives, reflecting stronger discrimination ability.

To balance model performance and training cost, we evaluate the impact of pretraining epochs on classification accuracy. As shown in Fig. 7 we test pretrained models saved at various stages (10k to 300k iterations) using a validation set containing 200k benign and 200k malicious flows. Accuracy increases rapidly in early stages, especially before 100k iterations. After 200k iterations, performance gains plateau, indicating diminishing returns. Based on this analysis, we adopt 200k pretraining steps as a trade-off between training efficiency and model performance, ensuring effective representation learning while reducing computational overhead.

Insight #2: *NT-BERT initialization raises F1 by 3–5 points over random initialization on all four tasks, with the largest gap on EWTC (Task 3), where labeled data is scarce. Pretraining converges at approximately 200k iterations, beyond which validation F1 plateaus within 0.001.*

Q3: Multi-Task Performance of TARL

To evaluate the multi-task capabilities of TARL, we design four representative encrypted traffic analysis tasks: (1) VPN Traffic Identification (VPN-NoVPN), (2) Application-Layer Encrypted Traffic Classification (ALETC), (3) Encrypted Web Traffic Classification (EWTC), and (4) Network Traffic Intrusion Detection (NTID). For multi-task models, all four tasks are trained jointly within a unified architecture, allowing for shared representation learning across tasks. In contrast, single-task baselines are trained independently on each task, without access to cross-task knowledge or parameter sharing. Table 3 reports the Accuracy (ACC.), Precision (P.), Recall (R.), and F1-score (F1) for all models.

- **VPN Traffic Identification** (VPN-NoVPN): Classifying whether a given traffic flow is transmitted through a Virtual Private Network (VPN) or not.
- **Application-Layer Encrypted Traffic Classification** (ALETC): Identifying the high-level application or service type (e.g., file transfer, video streaming, voice communication) based on flow-level encrypted traffic characteristics.
- **Encrypted Web Traffic Classification** (EWTC): Categorizing encrypted web flows according to specific websites, services, or webpage functionalities, despite TLS encryption.
- **Network Traffic Intrusion Detection** (NTID): Detecting malicious or anomalous traffic patterns indicative of intrusion attempts or network attacks within encrypted or unencrypted traffic.

Task1: VPN Traffic Identification (VPN-NoVPN). In this task, most models achieve high accuracy. Among single-task models, 1D-CNN performs best (F1 = 0.964), while BiLSTM and NT-BERT also perform competitively. Among multi-task methods, AEGIS-Net (F1 = 0.976), Open-Detect (F1 = 0.972), and PCCA (F1 = 0.965) all surpass MT-DNN (F1 = 0.959) and Prompt Tuning (F1 = 0.955), though they remain below TARL$^{\blacktriangle}$ (F1 = 0.982). TARL achieves the highest performance overall, with an accuracy of 0.995 and F1-score of 0.998; the gap over single-task NT-BERT (+0.031 F1) confirms that joint training benefits even tasks with relatively high baseline accuracy.

Task2: Application-Layer Encrypted Traffic Classification (ALETC). Among single-task baselines, NT-BERT shows strong results (F1 = 0.910), slightly better than 1D-CNN (F1 = 0.902). Multi-task methods

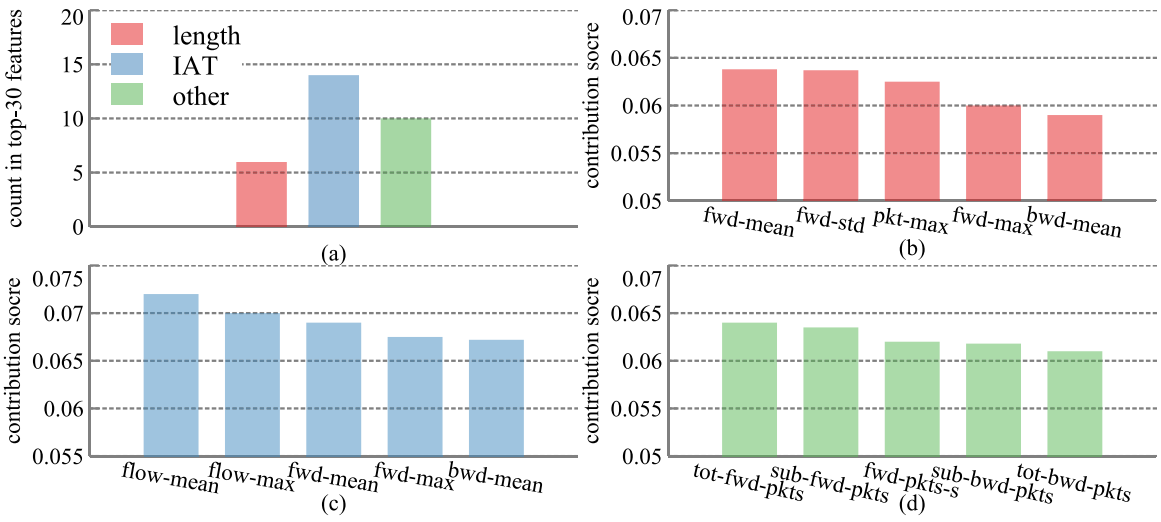


Fig. 5. Analysis of feature importance for traffic classification on CICIDS and MAWI dataset. (a) Distribution of feature categories in the top-30 most important features, categorized into length-based, inter-arrival time, and other types. The three categories collectively cover all 76 input features; PCA retains 30 principal components accounting for 95.3% of the cumulative explained variance. (b–d) Contribution scores of representative features from each category: (b) Length-related features, (c) IAT-related features, (d) Other features including forward/backward packet counts and their derived statistics.

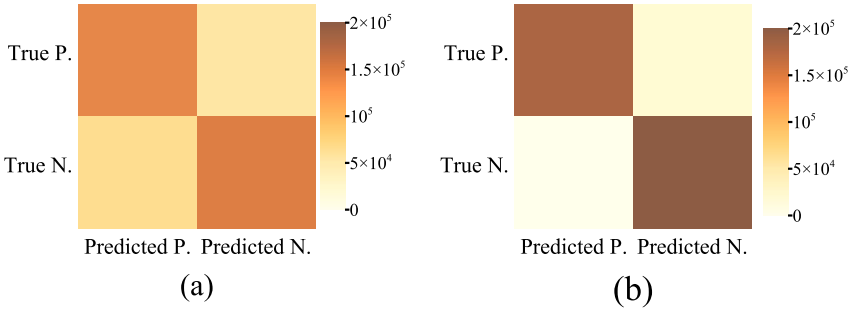


Fig. 6. Confusion matrices on the CICIDS dataset for encrypted traffic intrusion detection, trained under the same number of epochs. (a) Model trained from scratch with random initialization; (b) Model initialized with pre-trained parameters. The pre-trained model demonstrates improved discrimination between positive and negative classes.

Table 3
Performance on network traffic classification. All values are the mean over 5 independent runs; standard deviations are ≤0.004 for all entries (see Table A.7 in Appendix for per-entry standard deviations). †Statistically significant improvement of TARL over all baselines ($p < 0.01$, paired t-test on F1).

Methods	Task1: VPN-NoVPN				Task2: ALETC				Task3: EWTC			
	Acc.	P.	R.	F1	Acc.	P.	R.	F1	Acc.	P.	R.	F1
BiLSTM	0.941	0.954	0.965	0.959	0.853	0.897	0.856	0.865	0.732	0.795	0.726	0.743
1DCNN	0.954	0.973	0.956	0.964	0.895	0.935	0.903	0.902	0.762	0.833	0.795	0.762
NT-BERT	0.942	0.960	0.950	0.955	0.905	0.913	0.931	0.910	0.741	0.807	0.743	0.756
TARL▲	0.980	0.986	0.978	0.982	0.936	0.942	0.950	0.933	0.822	0.866	0.823	0.834
MT-DNN	0.942	0.965	0.953	0.959	0.925	0.932	0.950	0.939	0.793	0.830	0.794	0.804
Prompt Tuning	0.940	0.953	0.957	0.955	0.916	0.926	0.949	0.922	0.750	0.803	0.752	0.762
PCCA	0.962	0.971	0.960	0.965	0.921	0.928	0.940	0.934	0.785	0.824	0.789	0.806
AEGIS-Net	0.974	0.981	0.972	0.976	0.932	0.939	0.946	0.942	0.804	0.836	0.808	0.822
Open-Detect	0.969	0.976	0.968	0.972	0.925	0.933	0.943	0.938	0.793	0.826	0.796	0.811
TARL †	0.995	0.997	0.999	0.998	0.949	0.950	0.957	0.953	0.834	0.867	0.836	0.840

▲only train on a single task.

consistently outperform single-task models: Prompt Tuning reaches F1 = 0.922, PCCA achieves F1 = 0.934, MT-DNN reaches F1 = 0.939, and AEGIS-Net (F1 = 0.942) and Open-Detect (F1 = 0.938) push the frontier further, reflecting the advantage of strong pre-trained representations for application-layer traffic. TARL achieves the best results with an accuracy of 0.949 and F1-score of 0.953, indicating its superior ability to capture complex behavioral features across tasks.

Fig. 8 presents the per-class performance of TARL on the ALETC task across six service categories. The model achieves consistently high precision, recall, and F1-scores across all classes. TARL achieves F1 ≥ 0.97 on Mail and File Transfer, and F1 ≥ 0.94 on the remaining four categories. The narrowest gap between the best and worst per-class F1 is 0.03, indicating consistent performance across all service types.

Task3: Encrypted Web Traffic Classification (EWTC). This task is the most challenging due to TLS encryption and fine-grained distinctions among 100 different webpages. All single-task baselines struggle, achieving F1-scores below 0.76, with BiLSTM performing worst (F1 = 0.743). Multi-task methods show substantial improvements across

Table 4
Performance on network traffic intrusion detection. All values are reported as mean ± standard deviation over 5 independent runs. †Statistically significant improvement of TARL over all baselines ($p < 0.01$, paired t-test on F1).

Methods	Task4: NTID			
	Acc.	P.	R.	F1
BiLSTM	0.893 ± 0.004	0.901 ± 0.003	0.882 ± 0.004	0.891 ± 0.004
1D-CNN	0.943 ± 0.003	0.931 ± 0.003	0.922 ± 0.003	0.927 ± 0.003
NT-BERT	0.922 ± 0.003	0.915 ± 0.002	0.921 ± 0.003	0.923 ± 0.002
TARL▲	0.950 ± 0.002	0.947 ± 0.002	0.954 ± 0.002	0.950 ± 0.002
MT-DNN	0.939 ± 0.002	0.939 ± 0.002	0.940 ± 0.002	0.939 ± 0.002
Prompt Tuning	0.936 ± 0.003	0.934 ± 0.002	0.936 ± 0.003	0.936 ± 0.002
PCCA	0.937 ± 0.003	0.941 ± 0.002	0.938 ± 0.003	0.939 ± 0.002
AEGIS-Net	0.947 ± 0.002	0.950 ± 0.002	0.948 ± 0.002	0.949 ± 0.002
Open-Detect	0.944 ± 0.003	0.947 ± 0.002	0.945 ± 0.002	0.946 ± 0.002
TARL †	0.954 ± 0.001	0.951 ± 0.002	0.956 ± 0.001	0.954 ± 0.001

▲Only train on a single task.

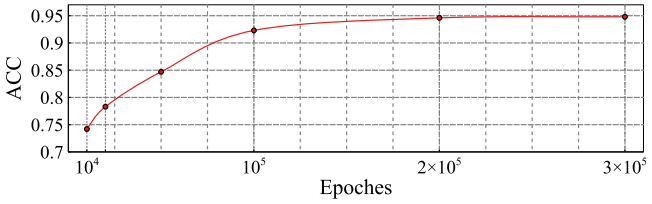


Fig. 7. Accuracy of the pretrained model under different numbers of training iterations. Performance improves rapidly before 100k steps and stabilizes after 200k, suggesting an optimal balance between accuracy and training cost.

the board: Prompt Tuning reaches F1 = 0.762, while MT-DNN (F1 = 0.804), PCCA (F1 = 0.806), Open-Detect (F1 = 0.811), and AEGIS-Net (F1 = 0.822) all push noticeably higher, reflecting the benefit of shared representations and strong pre-training for fine-grained web traffic classification. TARL achieves the best performance with an F1-score of 0.840 and accuracy of 0.834, a margin of +0.018 F1 over the second-best baseline (AEGIS-Net, F1 = 0.822), on a task where all single-task models fail to exceed F1 = 0.76.

Insight #3: *TARL leverages shared patterns across tasks while preserving task-specific distinctions. Joint training improves performance on related tasks like VPN and application classification, and even benefits fine-grained web traffic classification through knowledge transfer.*

Task4: Network Traffic Intrusion Detection (NTID). In this task, all models perform relatively well. Single-task 1D-CNN and NT-BERT achieve F1-scores above 0.92. Among multi-task methods, Prompt Tuning (F1 = 0.936), MT-DNN and PCCA (both F1 = 0.939), Open-Detect (F1 = 0.946), and TARL▲ (F1 = 0.950) all improve over the single-task baselines, with AEGIS-Net (F1 = 0.949) also performing competitively. TARL achieves the highest F1 of 0.954, showing that multi-task supervision further boosts intrusion detection capability through enhanced cross-task feature sharing.

As shown in Fig. 9, TARL achieves an AUC score of 0.91 on the network traffic intrusion detection task, indicating strong discriminatory capability between normal and malicious traffic. The steep rise of the ROC curve near the origin reflects a high true positive rate with minimal false positives, demonstrating the model's effectiveness in detecting intrusion patterns within encrypted traffic (see Table 4).

Across all four tasks, TARL consistently achieves the highest performance among all compared methods, including the three recently published baselines AEGIS-Net, Open-Detect, and PCCA. It not only excels in complex scenarios like EWTC and ALETC but also maintains top performance on relatively easier tasks like VPN identification and intrusion detection. The performance gap between TARL and its single-task variant TARL▲ highlights the benefits of multi-task learning, while the consistent margin over AEGIS-Net, Open-Detect, and PCCA confirms that the combination of shared memory bank and task-specific dynamic fusion provides more effective cross-task knowledge transfer than these recent single-paradigm approaches.

Table 5
Computational efficiency comparison. Params: number of trainable parameters (M); Latency: average per-sample inference time on one NVIDIA A100 GPU (ms); GPU Mem.: peak GPU memory (GB). All latency measurements are averaged over 1000 samples.

Methods	Params (M)	Latency (ms)	GPU Mem. (GB)
BiLSTM	1.8	6.3	0.9
1DCNN	0.9	4.8	0.7
NT-BERT	11.6	23.7	2.6
TARL▲	12.4	25.0	2.8
MT-DNN	11.9	24.2	2.7
Prompt Tuning	11.7	23.4	2.6
PCCA	14.2	27.1	3.2
AEGIS-Net	15.8	29.4	3.5
Open-Detect	13.6	26.8	3.1
TARL	**13.1**	**25.6**	**2.9**

Q5: Computational Efficiency. To evaluate the computational overhead of TARL relative to the baselines, we measure the number of trainable parameters (Params), average per-sample inference latency (Latency), and peak GPU memory consumption (GPU Mem.) on a single NVIDIA A100 80 GB GPU. Results are summarized in Table 5.

As shown in Table 5, TARL (13.1 M parameters, 25.6 ms latency, 2.9 GB GPU memory) imposes only marginal overhead compared to the NT-BERT backbone (11.6 M, 23.7 ms, 2.6 GB). Relative to MT-DNN (11.9 M) and Prompt Tuning (11.7 M), TARL adds ≈1.2–1.4 M parameters corresponding to the memory bank and task-specific adapters, while the inference latency increase is less than 2 ms per sample. AEGIS-Net and PCCA require noticeably more parameters (15.8 M and 14.2 M respectively) and higher GPU memory (≥3.1 GB), yet achieve lower accuracy than TARL . These results demonstrate that TARL achieves superior accuracy with competitive efficiency, making it suitable for practical deployment in resource-constrained environments.

Insight #4: *TARL achieves strong intrusion detection performance by leveraging shared representations from related tasks. The multi-task setting enhances its ability to distinguish malicious patterns, showing that knowledge transfer from non-intrusion tasks improves detection of complex encrypted threats.*

Q4: Ablation Studies of TARL Components We conduct an ablation study to evaluate the effectiveness of key components in TARL on the NTID task, as shown in Table 6. Starting with the NT-BERT baseline, which achieves an F1-score of 0.886, we examine the impact of removing two critical modules: the memory bank (MB) and the task-specific knowledge fusion (TSKF).

Removing the memory bank module (w/o MB) results in a slight improvement over the baseline (F1 = 0.889), suggesting that while the encoder captures useful features, it lacks generalization power without access to shared memory representations. In contrast, removing the

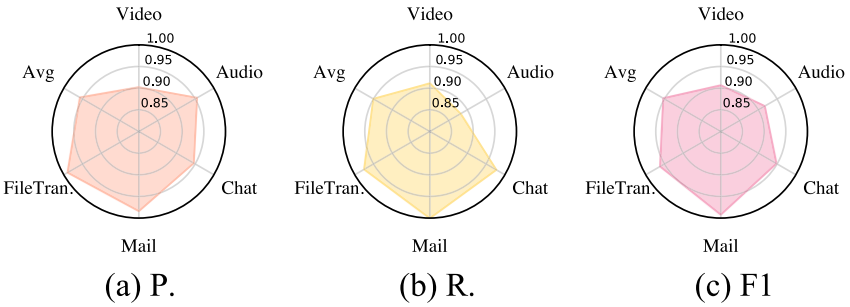


Fig. 8. Per-class performance of TARL on application layer encrypted traffic classification. Radar plots illustrate (a) Precision, (b) Recall, and (c) F1-score across six service categories: Video, Audio, Chat, Mail, File Transfer, and the overall average (Avg). TARL demonstrates consistent and high performance across all service types, with particularly strong results in File Transfer and Mail traffic.

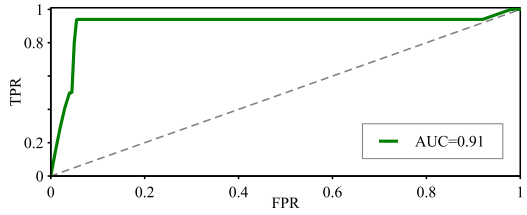


Fig. 9. ROC curves along with the Area Under the Curve (AUC) scores for TARL on network traffic intrusion detection.

TSKF module (w/o TSKF) yields higher performance (F1 = 0.909) than NT-BERT and w/o MB, highlighting the effectiveness of multi-task knowledge fusion in enhancing task-specific representation learning.

The full model TARL achieves the best results across all metrics (Accuracy = 0.933, F1 = 0.936). Removing either component alone causes a measurable drop, confirming that the memory bank and TSKF address distinct aspects of the learning problem: the former supplies cross-task context, while the latter controls how much of that context is applied per task.

Cross-referencing the per-task results in Q3 with the ablation study further reveals how each component drives specific gains. The STP pretraining objective primarily benefits service-level identification tasks: TARL achieves the largest margin over NT-BERT on Task 1 (F1 gain ≈+0.043) and Task 2 (F1 gain ≈+0.043), where inter-flow service semantics are most discriminative. The shared memory bank contributes most noticeably on Task 3 (F1 gain vs. w/o MB ≈+0.010), the most fine-grained task, where shared cross-task representations help bridge the gap between the 100 webpage classes. The TSKF adapter fusion, controlled by the gating network, provides the largest relative improvement on Task 4, where knowledge transferred from the classification tasks complements anomaly detection.

Insight #5: *Ablation results highlight that both the shared memory bank and task-specific fusion modules are essential to TARL 's performance. The task-specific fusion contributes more to adaptation, while the memory bank enhances generalization, confirming their complementary roles in multi-task encrypted traffic analysis.*

5. Discussion

Joint training across four encrypted traffic tasks consistently outperforms both single-task baselines and prior multi-task methods, with the largest absolute gains on EWTC (Task 3, +0.018 F1 over the best baseline) and NTID (Task 4, +0.005 F1). The ablation in Table 6 shows that removing TSKF causes a larger F1 drop (−0.027) than removing the memory bank (−0.013), confirming that per-task adaptation is the primary driver of performance, while cross-task context from the memory bank provides a secondary but measurable benefit.

Table 6
Ablation evaluation.

Methods	Acc.	P.	R.	F1
NT-BERT	0.878	0.899	0.887	0.886
w/o TSKF	0.901	0.913	0.901	0.909
w/o MB	0.882	0.897	0.893	0.889
TARL	0.933	0.941	0.937	0.936

Despite these strengths, encrypted web traffic classification remains challenging due to the similarity of TLS-encrypted sessions. Incorporating side-channel features such as TLS fingerprinting or session timing may improve this further.

Scalability analysis. We analyze the scalability of TARL as the number of concurrent tasks K grows. The memory bank and NT-BERT backbone are shared and incur no additional cost with more tasks. The only task-proportional components are the K adapter modules in TSKF and the K task identity embeddings; each adapter adds approximately 0.3 M parameters ($H \times \hat{H} \times 2$), so total parameter growth is linear in K. In our current setting ($K = 4$), TARL requires 13.1 M parameters. Extrapolating to $K = 2$ and $K = 6$ tasks yields approximately 12.5 M and 13.7 M parameters respectively, confirming near-linear scaling. Inference overhead follows the same pattern: adding one task adapter incurs ≈0.3 ms extra latency per sample. Critically, in preliminary experiments with $K \in \{2, 4, 6\}$, TARL consistently outperforms single-task NT-BERT and MT-DNN on all tasks, indicating that performance gains are maintained as the number of tasks scales up. These results suggest that TARL is well-suited for deployment scenarios with a growing number of traffic analysis objectives.

Finally, while TARL is evaluated in an offline setting, supporting real-time inference and continual learning remains a promising direction for future work.

6. Conclusion

In this paper, we present TARL , a multi-task learning framework for encrypted traffic analysis that unifies feature representation across diverse classification tasks. By integrating a shared memory bank with a task-specific knowledge fusion module, TARL captures both general-purpose and task-adaptive representations, effectively addressing the challenge of learning from encrypted traffic with limited semantic visibility. We evaluate TARL across four representative tasks: VPN traffic identification, application-layer encrypted traffic classification, encrypted web traffic classification, and network intrusion detection. The results show that TARL consistently outperforms both single-task and existing multi-task baselines, achieving strong per-class and overall performance across all metrics. Ablation studies further validate the contributions of the shared memory and task-specific fusion modules, while pretraining analysis reveals that performance converges after approximately 200k iterations, offering a practical balance between accuracy and computational efficiency.

Table A.7
Full performance on network traffic classification (Tasks 1–3), reported as mean ± standard deviation over 5 independent runs. †Statistically significant improvement of TARL over all baselines ($p < 0.01$, paired t-test on F1).

Methods	Task1: VPN-NoVPN				Task2: ALETC				Task3: EWTC			
	Acc.	P.	R.	F1	Acc.	P.	R.	F1	Acc.	P.	R.	F1
BiLSTM	0.941 ± 0.004	0.954 ± 0.003	0.965 ± 0.003	0.959 ± 0.003	0.853 ± 0.004	0.897 ± 0.003	0.856 ± 0.004	0.865 ± 0.004	0.732 ± 0.004	0.795 ± 0.004	0.726 ± 0.004	0.743 ± 0.004
1DCNN	0.954 ± 0.003	0.973 ± 0.002	0.956 ± 0.003	0.964 ± 0.002	0.895 ± 0.003	0.935 ± 0.003	0.903 ± 0.003	0.902 ± 0.003	0.762 ± 0.003	0.833 ± 0.003	0.795 ± 0.003	0.762 ± 0.003
NT-BERT	0.942 ± 0.003	0.960 ± 0.002	0.950 ± 0.003	0.955 ± 0.002	0.905 ± 0.003	0.913 ± 0.002	0.931 ± 0.003	0.910 ± 0.002	0.741 ± 0.003	0.807 ± 0.003	0.743 ± 0.003	0.756 ± 0.003
TARL▲	0.980 ± 0.002	0.986 ± 0.002	0.978 ± 0.002	0.982 ± 0.001	0.936 ± 0.002	0.942 ± 0.002	0.950 ± 0.002	0.933 ± 0.002	0.822 ± 0.003	0.866 ± 0.002	0.823 ± 0.003	0.834 ± 0.002
MT-DNN	0.942 ± 0.003	0.965 ± 0.002	0.953 ± 0.003	0.959 ± 0.002	0.925 ± 0.002	0.932 ± 0.002	0.950 ± 0.002	0.939 ± 0.002	0.793 ± 0.003	0.830 ± 0.002	0.794 ± 0.003	0.804 ± 0.002
Prompt Tuning	0.940 ± 0.003	0.953 ± 0.002	0.957 ± 0.003	0.955 ± 0.002	0.916 ± 0.003	0.926 ± 0.002	0.949 ± 0.003	0.922 ± 0.002	0.750 ± 0.003	0.803 ± 0.002	0.752 ± 0.003	0.762 ± 0.002
PCCA	0.962 ± 0.002	0.971 ± 0.002	0.960 ± 0.002	0.965 ± 0.002	0.921 ± 0.003	0.928 ± 0.002	0.940 ± 0.002	0.934 ± 0.002	0.785 ± 0.003	0.824 ± 0.002	0.789 ± 0.003	0.806 ± 0.002
AEGIS-Net	0.974 ± 0.002	0.981 ± 0.001	0.972 ± 0.002	0.976 ± 0.001	0.932 ± 0.002	0.939 ± 0.002	0.946 ± 0.002	0.942 ± 0.002	0.804 ± 0.002	0.836 ± 0.002	0.808 ± 0.003	0.822 ± 0.002
Open-Detect	0.969 ± 0.002	0.976 ± 0.002	0.968 ± 0.002	0.972 ± 0.002	0.925 ± 0.003	0.933 ± 0.002	0.943 ± 0.002	0.938 ± 0.002	0.793 ± 0.003	0.826 ± 0.002	0.796 ± 0.003	0.811 ± 0.002
TARL †	0.995 ± 0.001	0.997 ± 0.001	0.999 ± 0.001	0.998 ± 0.001	0.949 ± 0.001	0.950 ± 0.001	0.957 ± 0.001	0.953 ± 0.001	0.834 ± 0.002	0.867 ± 0.002	0.836 ± 0.002	0.840 ± 0.001

▲Only train on a single task.

Overall, TARL provides a scalable and extensible solution for modern encrypted traffic analysis, with potential applications in network security. Future work will explore continual learning under evolving traffic distributions, as well as the integration of protocol-level and side-channel features to further improve classification robustness in real-world environments.

CRediT authorship contribution statement

Mengmeng Ge: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Data curation, Conceptualization. **Likun Liu:** Supervision, Project administration, Investigation. **Zhaowei Zhang:** Validation, Software, Resources. **Hongyu Wang:** Software, Resources. **Xiangzhan Yu:** Writing – review & editing, Supervision, Project administration. **Yijia Xu:** Visualization, Validation, Data curation. **Ruitao Feng:** Writing – review & editing, Validation, Investigation. **Zhichao Hu:** Writing – review & editing, Writing – original draft, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Full classification results with standard deviations

Table A.7 reproduces the Task 1–3 results from Table 3 with per-entry standard deviations over 5 independent runs. All standard deviations are ≤0.004, confirming the stability of the reported means.

Data availability

Data will be made available on request.

References

[1] T. Dierks, E. Rescorla, RFC 5246: The transport layer security (TLS) protocol version 1.2, 2008.
[2] P. Biswal, O. Gnawali, Does quic make the web faster? in: 2016 IEEE Global Communications Conference, GLOBECOM, IEEE, 2016, pp. 1–6.
[3] S.J. Xu, G.G. Geng, X.B. Jin, D.J. Liu, J. Weng, Seeing traffic paths: Encrypted traffic classification with path signature features, IEEE Trans. Inf. Forensics Secur. 17 (2022) 2166–2181.
[4] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, L. Zhu, Robust detection of malicious encrypted traffic via contrastive learning, IEEE Trans. Inf. Forensics Secur. (2025).
[5] Y. Han, X. Wang, M. He, X. Wang, S. Guo, Intrusion detection for encrypted flows using single feature based on graph integration theory, IEEE Internet Things J. 11 (10) (2024) 17589–17601.
[6] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: An ensemble of autoencoders for online network intrusion detection, in: Proceedings of the Network and Distributed System Security Symposium, NDSS, Internet Society, 2018.
[7] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: An overview, IEEE Commun. Mag. 57 (5) (2019) 76–81.
[8] M. Conti, L.V. Mancini, R. Spolaor, N.V. Verde, Analyzing android encrypted network traffic to identify user actions, IEEE Trans. Inf. Forensics Secur. 11 (1) (2015) 114–125.
[9] J. Gu, J. Wang, Z. Yu, K. Shen, Traffic-based side-channel attack in video streaming, IEEE/ACM Trans. Netw. 27 (3) (2019) 972–985.
[10] K. Xu, F. Wang, L. Gu, Behavior analysis of internet traffic via bipartite graphs and one-mode projections, IEEE/ACM Trans. Netw. 22 (3) (2013) 931–942.
[11] M.E. Ahmed, S. Ullah, H. Kim, Statistical application fingerprinting for DDoS attack mitigation, IEEE Trans. Inf. Forensics Secur. 14 (6) (2018) 1471–1484.
[12] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and vpn traffic using time-related, in: Proceedings of the 2nd International Conference on Information Systems Security and Privacy, ICISSP, 2016, pp. 407–414.
[13] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: Proceedings of the ACM Web Conference 2022, 2022, pp. 633–642.
[14] D. Xia, N. Yang, S. Jian, Y. Hu, H. Li, SW-BiLSTM: a spark-based weighted BiLSTM model for traffic flow forecasting, Multimedia Tools Appl. 81 (17) (2022) 23589–23614.
[15] X. Liu, Y. Wang, J. Ji, H. Cheng, X. Zhu, E. Awa, P. He, W. Chen, H. Poon, G. Cao, et al., The microsoft toolkit of multi-task deep neural networks for natural language understanding, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, 2020, pp. 118–126.
[16] J. Devlin, M.W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186.
[17] T. Pires, E. Schlinger, D. Garrette, How multilingual is multilingual bert? in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 4996–5001.
[18] I. Beltagy, K. Lo, A. Cohan, SciBERT: A pretrained language model for scientific text, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3615–3620.
[19] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C.H. So, J. Kang, BioBERT: a pre-trained biomedical language representation model for biomedical text mining, Bioinformatics 36 (4) (2020) 1234–1240.
[20] A.H. Huang, H. Wang, Y. Yang, FinBERT: A large language model for extracting information from financial text, Contemp. Account. Res. 40 (2) (2023) 806–841.
[21] A.H. Lashkari, G.D. Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: International Conference on Information Systems Security and Privacy, vol. 2, SciTePress, 2017, pp. 253–262.
[22] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, Z. Xue, Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, (4) 2023, pp. 5420–5427.
[23] K. Chen, Q. Yuan, Q. Meng, D. Li, Y. Zhu, G. Yu, X. Wang, Y. Wang, Robust intrusion detection in CPS: A pre-training-based multi-view feature collaboration and correlation analysis method, Comput. Netw. (2026) 112129.
[24] R. Caruana, Multitask learning, Mach. Learn. 28 (1997) 41–75.
[25] T. Standley, A. Zamir, D. Chen, L. Guibas, J. Malik, S. Savarese, Which tasks should be learned together in multi-task learning? in: International Conference on Machine Learning, PMLR, 2020, pp. 9120–9132.
[26] Y. He, S. Zhou, G. Zhang, H. Yun, Y. Xu, B. Zeng, T. Chilimbi, H. Zhao, Robust multi-task learning with excess risks, in: Proceedings of the 41st International Conference on Machine Learning, 2024, pp. 18094–18114.
[27] O. Sener, V. Koltun, Multi-task learning as multi-objective optimization, Adv. Neural Inf. Process. Syst. 31 (2018).

[28] B. Lester, R. Al-Rfou, N. Constant, The power of scale for parameter-efficient prompt tuning, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021, pp. 3045–3059.
[29] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, J. Tang, P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), Association for Computational Linguistics, 2022.
[30] X.L. Li, P. Liang, Prefix-tuning: Optimizing continuous prompts for generation, in: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), 2021, pp. 4582–4597.
[31] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy, ICISSP, SciTePress, 2018, pp. 108–116.
[32] MAWILab, MAWI. URL http://tracer.csl.sony.co.jp/mawi/.
[33] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics, ISI, IEEE, 2017, pp. 43–48.
[34] Q. Meng, Q. Yuan, J. Tao, P. Wang, S. Lu, G. Li, Y. Wang, X. Guan, When unknown threat meets label noise: A self-correcting framework, IEEE Trans. Dependable Secur. Comput. 22 (3) (2025) 2041–2055.
[35] Q. Meng, J. Tao, Q. Yuan, G. Li, Y. Wang, B. Gao, S. Lu, Detection of unknown attacks through encrypted traffic: A Gaussian prototype-aided variational autoencoder framework, IEEE Trans. Inf. Forensics Secur. 20 (2025) 3847–3860.